Lab 2: Shell Setup & Command Sequences

Since working with a Unix system requires spending time in the shell it is desirable to make that work easy, engaging, and convenient. The default Unix shell environment does not provide a lot of comfort or information. We're going to modify configuration files in our home directory to customize the Unix shell to our own liking. We learn about important configuration files along the way and how we can custom changes to affect the way the shell behaves and what information is displayed to us.

Shell Setup: Making ourselves at $HOME

Configuration files are either system-wide (applying to all users) or per-user (in their home-directories). An example for such a system wide file is the list of shells available in the system. It is located in `/etc/shells`. Use `cat` to figure out which shells are available in this system. A user can set their own shell, which may be different from shells that other users work with. To distinguish these individual settings, configuration files in the user's home directory are processed once the user has successfully logged in.

When a Unix user is set up, a shell is defined (otherwise a user can't log in) for them by the administrator. In our department, `/bin/bash` is used, but users can still override that choice. In this lab, we'll switch to the zsh (Z shell). This enables us to get to know configuration files and modify them to our custom settings.

Switching to ZSH

Since the bash shell is provided, we're modifying one of its configuration files to switch to zsh right after login. This makes it look like we're using `zsh` from the start and there is no noticeable delay during the switch.

Configuration files are typically hidden from a regular `ls` output, which is why we learned how to display hidden files in Lab 1 using `ls -a`. The configuration file that gets parsed early during the login process for `bash` is called `.bash_profile`. Don't forget the leading `.` or the file won't be recognized by bash as it's configuration file. Create or modify it (using `ee` or `vim`), adding the following content:

```
if [ -t 1 ] && [ -x /usr/local/bin/zsh ] && [ "$SHELL" !=
↪  "/usr/local/bin/zsh" ]; then
  export SHELL="/usr/local/bin/zsh"
  exec /usr/local/bin/zsh
fi
```

The script does the following:

- `[ -t 1 ]` ensures the script only runs in interactive terminals, preventing issues with non-interactive logins.
- `[ -x /usr/local/bin/zsh ]` checks if Zsh is installed and executable.
- `[ "$SHELL" != "/usr/local/bin/zsh" ]` avoids recursion if Zsh is already the current shell.
- `exec /usr/local/bin/zsh` starts Zsh, replacing the current Bash shell for that session.

Once you've made these changes, log out of the system and back in. If all goes well, you'll be greeted by the zsh configuration wizard. You can exit it by pressing 0. This sets up a basic zsh configuration file for us with nothing but a comment in it. We're doing the configuration next anyway. Check the contents of the new `.zshrc`.

Check if the shell change was a success by executing these commands:

```
echo $SHELL
echo $0
```

It should list the path to the shell in both cases. Congratulations, you've switched to ZSH. Now let's make it useful. Push this file to the `Lab 2` directory in your repository with a meaningful commit message. You can always retrieve this file from the repository if you accidentally delete it or it does not work anymore and you want to start over.

ZSH Configuration

You've probably noticed by now that the aliases we defined during the Game in Lab 1 are no longer working after logging out and back into the system. The reason is that alias definitions on the shell are only active as long as that shell instance exists. To make these aliases available each time right after logging in, we add them to our shell configuration file `.zshrc`.

To keep things orderly, create a comment (starting with a #) on a new line that's called `Aliases`. That way, we can keep all the aliases we define together to find them quickly when looking at the file. Add the following aliases:

```
alias ll='ls -l'
alias la='ls -A'
```

Here are some git-related aliases that may help you working with the version control system:

```
# Git-related aliases
alias gs='git status'
alias gl='git log --oneline --graph --all'
alias gp='git pull'
alias gc='git commit -v'
```

To apply the changes to your current session without logging out of the system and back in, use the `source` command followed by the file name to read. You should now have your new aliases available. Add your own custom aliases that you defined as part of the solution in Lab 1 and repeat the above steps.

Great, add this file to your repository as well by making another commit.

Next up, we'll add some environment variables. These variables define common information that the shell reads or scripts make use of. Run the `env` command to see what kind of variables are currently defined. Some of them are provided by the system, but you can change or even add your own to that list. Add a new section `# Environment Variables` to your `.zshrc`:

```
export EDITOR=ee
export PATH="$HOME/bin:$PATH"
```

Explanation:

`export EDITOR=ee` ensure that commands relying on an editor, such as `git commit`, open in `ee`. The `PATH` variable lets users add custom directories, making any executable scripts in `$HOME/bin` accessible from anywhere. Create the `bin` directory using `mkdir`. We'll use this later when we do programming with the shell.

The shell itself is configured with this file and has it's own lines defined in the documentation. To keep a history of your commands typed in the shell, a file called `.zsh_history` is maintained. It's size and location is defined by the following entries:

```
# Enable history saving for command recall
HISTSIZE=1000            # Commands stored in memory
SAVEHIST=1000            # Commands saved to history file
HISTFILE=~/.zsh_history  # Location of history file
```

Use the `source` command again to apply the changes. Push this file to your repo with a commit message describing the change you've made.

Powerlevel 10K



The changes we made so far are useful, but we still have a rather plain black and white screen. We're going to change that by configuring a custom shell prompt which contains some extra information. To set it up, we first need to add the functionality to our home directory by cloning the repository like this:

```
git clone --depth=1 https://github.com/romkatv/powerlevel10k.git
 ~/powerlevel10k
```

This clones the GitHub repository and stores it in a new local directory in called `powerlevel10k`. Next, call the theme configuration from the end of your `.zshrc` file:

```
source ~/powerlevel10k/powerlevel10k.zsh-theme
```

This loads the theme each time your shell starts as part of the login process. To configure your shell prompt, run the configuration wizard:

```
p10k configure
```

You're asked a series of questions. Pick and chose the options that you like best (your shell, your choices). If you've changed your mind to want to see how different settings look like, run the above command again. When satisfied, create a screenshot of your new shell and put it next to the configuration files in your `Lab 2` directory in the repository.

As more configuration files get added to your home directory, they tend to clutter it. To keep things sorted, the XDG standard was defined. Among other things, it defines that there is a separate conifguration directory `.config` with a subdirectory for each software that holds its specific configuration. Create a directory called `.config/zsh` using `mkdir`. Look at the man page to find an option to create multiple directories with a single command line. We're now adding Plugins to ZSH into that directory.

Adding Plugins to ZSH

Plugins extend the functionality of ZSH. There are plenty of ready-made configurations that provide a lot of plugins. A popular one called `oh-my-zsh` has a lot of functionality, but is too much for our needs and slows down shell execution somewhat. Typically, we only need two: `zsh-autosuggestions` and `zsh-syntax-highlighting`.

- `zsh-autosuggestions`: provides real-time command suggestions based on your user's shell history and frequently used commands. As you start typing, Zsh suggests potential completions for the command, displayed in a lighter color. Accept the suggestion by pressing the → (right arrow) key or continue typing to refine the command.
- `zsh-syntax-highlighting`: provides real-time syntax highlighting for commands as they are typed. Correct syntax appears in a standard color, while invalid commands or options are highlighted in red, making it easy to spot errors before running commands. This visual feedback helps you learn and understand command structure and flags more intuitively.

We'll set these plugins up as follows. First we start with `zsh-autosuggestions`.

1. `git clone https://github.com/zsh-users/zsh-autosuggestions ~/.config/zsh/zsh-autosuggestions`

We're cloning the repository for the plugin into our zsh-specific configuration directory below `.config`.

2. Edit `.zshrc` and add the following line at the end:

```
source ~/.config/zsh/zsh-autosuggestions/zsh-autosuggestions.zsh
```

This starts the plugin as part of your shell startup after login.

3. Apply the changes again using `source ~/.zshrc`

Start typing a command you typed before (like `source`). Notice how the full command is suggested in a grey color? Use the right arrow key or press Ctrl+e to accept the completion. Hit enter to execute it. This is a big time-safer, especially with long and complicated commands that have a lot of parameters.

Commit those changes to your repository as you did before.

We set up `zsh-syntax-highlighting` in a similar way.

1. Clone the plugin:

```
git clone https://github.com/zsh-users/zsh-syntax-highlighting.git
  ~/.config/zsh/zsh-syntax-highlighting
```

2. In your `.zshrc` file, add the following line to load the plugin after the other plugin:

```
source
 ↪  ~/.config/zsh/zsh-syntax-highlighting/zsh-syntax-highlighting.zsh
```

3. Apply the changes to the current session: `source ~/.zshrc`

Typing `lx -l` would highlight the command in red, indicating it does not exist, while `ls -l` would appear in green, showing it's a valid command. This helps you identify whether your current command line is valid as you type.

Push these changes in the `.zshrc` to your repository.

Configuring `.gitconfig`

The `.gitconfig` file is a central configuration file for Git. Create the file if it does not exist yet. In this part, you will configure your name and email address to ensure the commits you make are properly attributed to you.

Follow these steps:

1. View your current configuration: Check if your name and email are already set:

```
git config --global --list
```

2. If not, set your name and email address like this:

```
git config --global user.name "Your Full Name"
git config --global user.email "your.email@domain.com"
```

3. Repeat step 1 to verify that the changes are now reflected in the output. Check the contents of the file `~/.gitconfig` and how they are structured. This file represents the global git configuration. Each working copy may maintain its own configuration that overrides the global config. But some settings are universally applicable to all repositories you work with. For that reason, having the global configuration file keeping some general settings is a good strategy.

Signing Commits with SSH

Signed commits allow others to verify that you created the commit. This is useful for collaborative or sensitive projects. In this setup, you will use your existing SSH key that you registered with GitLab in Lab 1. The key used to sign your commits proves that you know the credentials to access and make changes to the repository. Do the following:

1. Verify that you already have your SSH key available, located in `~/.ssh` (check back with Lab 1 for details). Since you made commits for Lab 1 (the lab report), this works already. Make sure to add the key to the SSH agent before you continue.
2. Enable signing commits with your SSH key by extending your `.gitconfig` file like we did above:

```
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/nameofyourkey.pub
```

3. Create a signed commit. Make a change to a file in your repository (`lab2-docs.md` comes to mind) and `git add` it as before. The commit step looks like this now:

```
git commit -S -m "Signed commit example"
```

Push the change to the repository as normal. The `-S` flag often gets forgotten when doing a commit, which causes them to not get signed. A setting in the `.gitconfig` file ensures that commits are always signed:

```
git config --global commit.gpgsign true
```

4. When looking at the signature in the GitLab web UI, the commit got marked as `unverified`. Looking at the git log, there is a `no signature` indicator next to it:

```
git log --show-signature
```

The reason for this is that there is no way for git to verify the key against some known signature. We will fix this by adding a list of `allowed_signers` locally.

5. Create a file called `~/.ssh/allowed_signers`. It contains on each line the signatures of emails along with their SSH keys that you trust (i.e. your own to start with). Here is an example line:

```
example@code.fbi.h-da.de namespaces="git" ssh-ed25519 SSH-Public-Key-ID
```

To add your own key to the `allowed_signers` run the following command to extract your email and existing public key:

```
echo "$(git config --get user.email) namespaces=\"git\" $(cat
↪  ~/.ssh/<MY_KEY>.pub)" >> ~/.ssh/allowed_signers
```

6. Make the `allowed_signers` file known to git to verify commits when pushing them to the repository. Run the following command to add a reference to the file to `.gitconfig`:

```
git config --global gpg.ssh.allowedSignersFile "~/.ssh/allowed_signers"
```

This should be enough to provide git with the necessary information. Run

```
git log --show-signature
```

again to see the "Good 'git' signature" next to the signed commits. The GitLab web UI will also show a green `Verified` label now. Ensure that the `.git/config` file in your working directory does not contain any entries that contradict the ones in `.gitconfig`. In a project with more than one contributor, this file will hold specific information about this repository, along with the `allowed_signers` file to reference everyone's SSH public key for verification.

You can find more information about signing commits in GitLab in their help system: https://code.fbi.h-da.de/help/user/project/repository/signed_commits/ssh.md

The `.gitignore` file

The `.gitignore` file specifies intentionally untracked files that Git should ignore. These are files that you want to exclude (unnecessary or sensitive files) from version control, keeping repositories clean and focused.

Typical examples include:

- Temporary files
- Build artifacts
- System-specific files

How it Works:

Patterns in `.gitignore` match files or directories. Ignored files are not tracked by Git and won't be staged for commits. Place the `.gitignore` file in the root of a repository.

Add your own `.gitignore` file to your repository by following these steps:

1. In your repository, create the file:

```
touch .gitignore
```

2. Add Patterns: Open the file in a text editor and add patterns:

```
# Backup and temporary files
*~
*.swp
*.bak
*.tmp
*.old

# System files
.DS_Store
Thumbs.db
.trash/

# Logs and debugging files
*.log
*.out
*.err
*.pid

# Shell history files
.history
.bash_history
.zsh_history

# Compiled files and binaries
*.o
*.so
*.a
*.class
*.pyc

# Build directories
build/
tmp/
.cache/
```

```
dist/

# Application-specific files
node_modules/
.env
.idea/
.vscode/
```

You can also extend this with your own patterns.

Note: This won't affect files that are already in your repository as part of an earlier commit. Only new files that match this pattern get ignored.

3. Test Ignored Files: Create a file matching a pattern:

```
echo "Temporary data" > temp.log
```

Check the status of git:

```
git status
```

The file should not appear in the output.

4. Track the `.gitignore` file by adding it to your repository:

```
git add .gitignore
git commit
git push
```